

Lịch buổi tối (bản 3) — chuyển C tối thiểu, tận dụng tối đa

Nguyên tắc mới: chỉ viết lại bằng C những gì app dùng lúc chạy. Phần còn lại giữ nguyên.

1. Vạch lại ranh giới

Câu hỏi đúng không phải "code nào là C++" mà là "code nào chạy trong app?"

File	Chạy trong app?	Quyết định
dictionary.cpp — truy vấn DB	✓ mỗi lần gõ phím	→ C
kana.cpp — chuẩn hoá katakana	✓ mỗi lần tìm kiếm	→ C
entry.h — kiểu dữ liệu	✓	→ C
csvreader.cpp — parser CSV	✗ chỉ importer dùng	giữ C++
common.cpp — tìm cột theo tên	✗ chỉ importer dùng	giữ C++
importer/main.cpp	✗ công cụ dev, chạy 1 lần	giữ C++
entrymodel.cpp	✓ nhưng là cầu nối Qt	giữ C++
Main.qml, app/main.cpp	✓ giao diện	giữ nguyên

Importer không phải backend của app. Nó là công cụ build-time: chạy một lần để sinh ra dictionary.db, không đóng gói kèm sản phẩm. Viết lại parser CSV bằng C tốn trọn một tối mà không làm app "C hơn" chút nào.

Đây cũng là cách chia chuẩn trong doanh nghiệp: code sản phẩm theo ngôn ngữ bắt buộc, công cụ nội bộ dùng gì tiện thì dùng.

Câu nói với thầy

Lỗi chạy trong app — truy vấn database, xử lý Unicode, chuẩn hoá kana — em viết hoàn toàn bằng C, gọi thẳng sqlite3 C API, không dùng Qt SQL. Thư viện dict-core không link Qt gì cả, và importer.exe chứng minh điều đó vì nó chạy được mà không cần DLL Qt nào cho phần database.

Riêng công cụ import chạy một lần lúc build thì em giữ Qt cho phần đọc CSV, nhưng nó vẫn ghi database qua đúng API C đó.

Nếu thầy muốn toàn bộ kể cả importer là C, thì thêm một tối cho dict_csv.c — có sẵn ba tối dự phòng.

Cái được

	Bản cũ	Bản này
Tối cho việc chuyển đổi	4	2
File C phải viết mới	5	3 (2 đã xong)
Tối dự phòng cuối chạng	1	3
Mốc khô phục	18/8	16/8

2. Đã làm xong (tối 14/8)

- ✓ core/dict_types.h + dict_types.c — struct, mã lỗi, DictEntryList với init/push/free
- ✓ core/dict_utf8.h + dict_utf8.c — decode/encode UTF-8, đã test ㄱㅡㅁ → 3053 30fc 3072 30fc
- ✓ CMakeLists đã có LANGUAGES C CXX, C11, sqlite3 amalgamation build sạch
- ✓ Bug FTS5 trong bản C++ đã sửa

Còn nợ: kiểm tra (b1 & 0xC0) != 0x80 ở nhánh 2 và 4 byte của dict_utf8_decode.

3. Cấu trúc thư mục đích

```
D:\FPT\FPTJapan\Dictionary\
├─ CMakeLists.txt
├─ core\
│   ├── dict_types.h/.c          ← C THUẦN, không header Qt nào
│   ├── dict_utf8.h/.c          ✓ xong
│   ├── dict_kana.h/.c          ✓ xong
│   ├── dict_db.h/.c            ← tối 15/8
│   ├── dict_deinfect.h/.c      ← tối 15+16/8
│   ├── dict_deinfect.h/.c      ← tối 21+22/8
│   └─ third_party\sqlite3.c, sqlite3.h ✓ xong
├─ tool\importer\
│   ├── main.cpp                ← C++ + Qt, công cụ build-time
│   ├── csvreader.h/.cpp        (giữ, chỉ đổi phần ghi DB)
│   ├── common.h/.cpp           (chuyển từ core\, không sửa)
│   └─ Main.qml                 (chuyển từ core\, không sửa)
└─ app\
    ├── entrymodel.h/.cpp       ← C++ + Qt, giao diện
    ├── main.cpp
    └─ Main.qml
```

Thước đo: thấy #include <Q...> trong file .c là sai chỗ.

Xoá được sau khi bản C chạy: core/entry.h, core/kana.h/.cpp, core/dictionary.h/.cpp.

Ba file csvreader, common chỉ **di chuyển** sang tool\importer\, không sửa một dòng.

4. CMakeLists.txt đích

```
cmake_minimum_required(VERSION 3.16)
project(Dictionary VERSION 0.1 LANGUAGES C CXX)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(QT_QML_GENERATE_QMLLS_INI ON)

# Không còn Sql – ca du an không dùng Qt SQL nữa
find_package(Qt6 REQUIRED COMPONENTS Core Quick)
qt_standard_project_setup(REQUIRES 6.8)

# ----- Loi C thuan: KHONG link Qt gi ca -----
add_library(dict-core STATIC
    core/dict_types.h core/dict_types.c
    core/dict_utf8.h core/dict_utf8.c
    core/dict_kana.h core/dict_kana.c
    core/dict_db.h core/dict_db.c
    core/third_party/sqlite3.c
)
target_include_directories(dict-core PUBLIC core core/third_party)

if(UNIX)
    target_link_libraries(dict-core PRIVATE dl pthread m)
endif()

set_source_files_properties(core/third_party/sqlite3.c
    PROPERTIES COMPILE_FLAGS "-w")

# ----- App QML -----
qt_add_executable(appDictionary app/main.cpp)

qt_add_qml_module(appDictionary
    URI Dictionary
    QML_FILES app/Main.qml
    SOURCES app/entrymodel.h app/entrymodel.cpp
)

set_target_properties(appDictionary PROPERTIES
    MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
    MACOSX_BUNDLE_SHORT_VERSION_STRING ${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
    MACOSX_BUNDLE TRUE
    WIN32_EXECUTABLE TRUE
)

target_link_libraries(appDictionary PRIVATE Qt6::Quick dict-core)

install(TARGETS appDictionary
    BUNDLE DESTINATION .
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)

# ----- Importer: C++ doc CSV, ghi DB qua API C -----
qt_add_executable(importer
    tool/importer/main.cpp
    tool/importer/csvreader.h tool/importer/csvreader.cpp
    tool/importer/common.h tool/importer/common.cpp
)
target_link_libraries(importer PRIVATE Qt6::Core dict-core)
set_target_properties(importer PROPERTIES WIN32_EXECUTABLE FALSE)
```

Điểm đáng chú ý: find_package không còn Sql. Toàn dự án không dùng Qt SQL nữa — mọi thao tác database đi qua dict_db.c. Đó là bằng chứng mạnh nhất cho thấy.

5. Chuẩn code C (giữ nguyên từ bản 2)

Đặt tên: hàm công khai `dict_<module>_<hành_động>`; hàm nội bộ `static`; kiểu PascalCase có tiền tố; macro UPPER_SNAKE; biến snake_case.

Header: include guard `#ifndef`, không dùng `#pragma once`. Bọc `extern "C"` toàn bộ nội dung — mở sau include, đóng trước `#endif` cuối.

Báo lỗi: trả mã `DictStatus`, dữ liệu qua tham số ra.

```
/* DUNG */
int dict_db_search(DictDb *db, const char *q, int limit, DictEntryList *out);
```

Opaque struct: `typedef struct DictDb DictDb;` trong `.h`, định nghĩa đầy đủ trong `.c`. Hệ quả: `dict_db.h` không cần include `sqlite3.h`.

Sở hữu bộ nhớ: ghi vào comment header — hàm nào cấp phát thì có hàm `_free` đi kèm.

Chuỗi: chỉ `snprintf`. Cấm `strcpy`, `strcat`, `sprintf`.

Giữ đường lui sang .cpp: ép kiểu tường minh cho `malloc/realloc`, `(void)` cho hàm không tham số.

Kích thước buffer (đo trên `data.xlsx`): word 64, reading 64, romaji 32, pos 32, meaning 128, english 192, level 8. Dài nhất thực tế lần lượt là 42, 51, 26, 18, 81, 145, 2.

6. Lịch 13 tối

Tối 15/8 — Kana + ghi database

Phần 1: dọn thư mục (~20 phút)

```
mkdir tool\importer # đã có
git mv core/csvreader.h core/csvreader.cpp tool/importer/
git mv core/common.h core/common.cpp tool/importer/
mkdir app
git mv core/entrymodel.h core/entrymodel.cpp app/
git mv main.cpp Main.qml app/
```

Sửa CMakeLists theo mục 4, build. Chưa có gì mới, mọi thứ cũ vẫn chạy.

Phần 2: core/dict_kana.c

```
/* Doi katakana sang hiragana. Ky tu khac giu nguyen.
 * Tra ve DICT_OK, hoac DICT_ERR_ARG neu dst khong du cho. */
int dict_kana_to_hiragana(const char *src, char *dst, size_t dst_size);
```

Duyệt bằng `dict_utf8_decode`, codepoint trong `[0x30A1, 0x30F6]` thì trừ `0x60`, `dict_utf8_encode` ghi vào `dst`. `—` (U+30FC) nằm ngoài khoảng nên tự động giữ nguyên.

Trước mỗi lần ghi phải kiểm tra còn đủ chỗ, kể cả 1 byte cho `'\0'` cuối.

Xong khi: コーヒー → コーヒー, 日本語 giữ nguyên.

Phần 3: core/dict_db.c phần ghi

```
typedef struct DictDb DictDb; /* opaque */

int dict_db_create(const char *path, DictDb **out); /* xoa file cu, tao schema */
void dict_db_close(DictDb *db);
const char *dict_db_last_error(const DictDb *db);

int dict_db_begin(DictDb *db);
int dict_db_commit(DictDb *db);
int dict_db_rollback(DictDb *db);
int dict_db_insert(DictDb *db, const DictEntry *e);
int dict_db_create_indexes(DictDb *db); /* nho COLLATE NOCASE */
int dict_db_create_fts(DictDb *db); /* tao VA nap du lieu */
```

sqlite3 C API: `sqlite3_open`, `sqlite3_exec`, `sqlite3_prepare_v2`, `sqlite3_bind_text`, `sqlite3_bind_null`, `sqlite3_step`, `sqlite3_reset`, `sqlite3_finalize`, `sqlite3_close`, `sqlite3_errmsg`.

Đọc <https://sqlite.org/cintro.html> — 20 phút.

Bốn bẫy - `sqlite3_bind_text(stmt, i, s, -1, SQLITE_TRANSIENT)` — `-1` là "tự đếm tới `\0`". `SQLITE_TRANSIENT` bảo sqlite copy chuỗi; `SQLITE_STATIC` với biến cục bộ là đọc bộ nhớ đã chết - Quên `sqlite3_reset()` sau mỗi `step()` trong vòng lặp → chỉ chèn được một dòng - Chỉ số bind bắt đầu từ **1**, không phải 0 - `level` rỗng phải `sqlite3_bind_null`

Phần 4: nối importer vào

`tool/importer/main.cpp` giữ nguyên phần đọc CSV. Chỉ thay khối Qt SQL bằng lời gọi C:

```
extern "C" {
#include "dict_db.h"
}

DictDb *db = NULL;
if (dict_db_create(dbPath.toUtf8().constData(), &db) != DICT_OK) {
    out << "Loi mo db" << Qt::endl;
    return 1;
}
dict_db_begin(db);

for (int r = headRow + 1; r < data.size(); ++r) {
    DictEntry e;
    memset(&e, 0, sizeof e);          /* quan trong: xoa rac truoc */

    QByteArray w = cell(row, iWord).toUtf8();
    snprintf(e.word, sizeof e.word, "%s", w.constData());
    /* ... 7 truong con lai ... */

    if (dict_db_insert(db, &e) != DICT_OK) {
        dict_db_rollback(db);
        return 1;
    }
}

dict_db_commit(db);
dict_db_create_indexes(db);
dict_db_create_fts(db);
dict_db_close(db);
```

Bây: QByteArray tạm chết ngay sau câu lệnh. Phải hứng vào biến (QByteArray w = ...) rồi mới lấy constData() , đừng viết gộp một dòng.

Xong khi: COUNT(*) = 14.657, khớp con số ghi tối 14/8. Có thời gian import mới để so với bản Qt.

Tối 16/8 — Đọc database + câu nối ⚠ MỐC KHÔI PHỤC

```
int dict_db_open(const char *path, DictDb **out);    /* mo file co san */
int dict_db_search(DictDb *db, const char *q, int limit, DictEntryList *out);
int dict_db_search_meaning(DictDb *db, const char *q, int limit, DictEntryList *out);
```

Đọc kết quả bằng sqlite3_column_text(stmt, i) — trả const unsigned char*, ép sang const char* rồi snprintf vào struct.

Nhớ giữ chi tiết từ bản C++: cột word so với chuỗi **gốc**, chỉ reading_hira mới so với bản đã chuyển hiragana.

app/entrymodel.cpp — file C++ duy nhất biết về DictEntry:

```
extern "C" {
#include "dict_db.h"
}

DictEntryList list;
QByteArray q = query.toUtf8();
if (dict_db_search(m_db, q.constData(), 100, &list) == DICT_OK) {
    for (int i = 0; i < list.count; ++i) {
        Entry e;
        e.word    = QString::fromUtf8(list.items[i].word);
        e.reading = QString::fromUtf8(list.items[i].reading);
        /* ... */
        m_entries.append(e);
    }
}
dict_entry_list_free(&list);    /* BAT BUOC */
```

QString::fromUtf8 chứ không phải QString(...) — cái sau dùng bằng mã hệ thống, chữ Nhật hỏng.

Tiêu chí dừng: hết tối nay app chưa chạy lại được thì quay về v0.1-cpp và báo thầy. Hoặc đổi .c → .cpp nếu vướng khâu link.

Xong khi: gõ あい trong app ra đúng 10 kết quả như đã ghi tối 14/8. **Việc chuyển đổi kết thúc ở đây.**

Xoá được: core/entry.h, core/kana.*, core/dictionary.*.

Tối 17/8 — Tìm kiếm tức thời + thông minh

QML: Timer 250ms debounce; LIMIT 100; trạng thái rỗng có nội dung (chưa gõ → gợi ý; không kết quả → "thử gõ romaji hoặc nghĩa tiếng Việt"); hiện số kết quả.

C: nhận diện kiểu chuỗi, dùng lại dict_utf8_decode:

```
typedef enum {
    DICT_INPUT_UNKNOWN, DICT_INPUT_KANJI, DICT_INPUT_HIRAGANA,
    DICT_INPUT_KATAKANA, DICT_INPUT_LATIN, DICT_INPUT_VIETNAMESE
} DictInputType;

DictInputType dict_kana_detect(const char *s);
```

Khoảng codepoint: kanji U+4E00–U+9FFF, hiragana U+3041–U+3096, katakana U+30A1–U+30F6. Latin có dấu tiếng Việt (U+1EA0–U+1EF9) → tra FTS5.

Xếp hạng bằng CASE WHEN trong ORDER BY: khớp chính xác → tiền tố → giữa chuỗi.

Tối 18/8 — Màn hình chi tiết + sao chép

- `StackView`: bấm vào từ → push sang trang chi tiết. Nút quay lại + phím Esc. Giữ nguyên kết quả tìm kiếm khi quay lại
- Trang chi tiết: `word` cỡ lớn, `reading`, `romaji`, `pos`, nghĩa Việt dạng danh sách đánh số, nghĩa Anh, `level`
- Tách meaning theo "; " bằng `QString::split` ở tầng bridge
- Nút sao chép: `Q_INVOKABLE void copyToClipboard(const QString&)` dùng `QGuiApplication::clipboard()`
- Phím tắt: Ctrl+F vào ô tìm, Esc xoá/quay lại, ↑ ↓ chọn, Enter mở chi tiết

Tối 19/8 — Lịch sử + yêu thích

Quyết định kiến trúc đáng kể lúc phỏng vấn: **tách DB hệ thống và DB người dùng**.

`dictionary.db` chỉ đọc, cạnh `exe`. `user.db` ghi được, ở `QStandardPaths::AppDataLocation` — cập nhật app không mất dữ liệu, và app cài ở `Program Files` vẫn ghi được.

Kỹ thuật: `ATTACH DATABASE` để `JOIN` qua hai file.

```
ATTACH DATABASE '<duong_dan>/user.db' AS userdb;

CREATE TABLE IF NOT EXISTS userdb.history (
    entry_id INTEGER PRIMARY KEY,
    viewed_at INTEGER NOT NULL
);
CREATE TABLE IF NOT EXISTS userdb.favorites (
    entry_id INTEGER PRIMARY KEY,
    added_at INTEGER NOT NULL
);

INSERT INTO userdb.history(entry_id, viewed_at) VALUES(?, ?)
ON CONFLICT(entry_id) DO UPDATE SET viewed_at = excluded.viewed_at;

SELECT w.* FROM userdb.history h
JOIN words w ON w.id = h.entry_id
ORDER BY h.viewed_at DESC LIMIT ?;
```

```
int dict_db_attach_user(DictDb *db, const char *user_db_path);
int dict_db_add_history(DictDb *db, int entry_id);
int dict_db_list_history(DictDb *db, int limit, DictEntryList *out);
int dict_db_clear_history(DictDb *db);
int dict_db_toggle_favorite(DictDb *db, int entry_id);
int dict_db_is_favorite(DictDb *db, int entry_id, int *out);
int dict_db_list_favorites(DictDb *db, int limit, DictEntryList *out);
```

QML: nút sao ở trang chi tiết; ô tìm rỗng thì hiện lịch sử thay vì màn hình trắng; trang xem yêu thích; nút xoá lịch sử.

Xong khi: tắt app mở lại vẫn còn dữ liệu, và `dictionary.db` không bị ghi vào.

Tối 20/8 — Bộ lọc + duyệt theo cấp

- `ComboBox` lọc theo `part_of_speech` (17 giá trị) và `level`
- Mục "Chưa phân loại" dùng `level IS NULL` (9.044 dòng)
- Duyệt theo cấp: ô tìm rỗng + chọn N3 → liệt kê từ N3. Dừng lại truy vấn lọc, bỏ điều kiện từ khoá
- Nút xoá hết bộ lọc, hiện số kết quả sau khi lọc
- Ghép `WHERE` động bằng chuỗi, nhưng **giá trị luôn bind bằng ?**

Bây: `= NULL` không bao giờ đúng, phải `IS NULL`.

Tối 21/8 — Deinflection phần 1

```
typedef struct {
    const char *from; /* duoi dang chia */
    const char *to; /* duoi dang goc */
    int type;
} DeinflectRule;

static const DeinflectRule RULES[] = {
    { "ました", "る", DICT_VERB_ICHIDAN },
    /* ... */
};

int dict_deinflect(const char *word, char out[][DICT_WORD_LEN], int max);
```

Mảng `static const` ở phạm vi file — không cấp phát động. Đây là chỗ C gọn hơn C++.

So khớp đuôi bằng `memcmp` trên byte cuối — đuôi là chuỗi byte cố định, không cần decode UTF-8.

Nổi vào luồng tìm: tra trực tiếp trước, không thấy mới lùi chia. Hiện "Dạng gốc của 食べました".

Đối chiếu: <https://www.sjfaq.org/cgi/verb-deinflect.cgi>

Xong khi: 食べました → 食べる, 見ない → 見る, 行きます → 行く.

Tối 22/8 — Deinflection phần 2

Âm tiện (音便): って → う/つ/る, んで → ん/ぶ/む, いて → く, いで → ぐ. Một đuôi ra nhiều gốc.
Tính từ đuôi い: かった, くない, くて, ければ. Thể khả năng, bị động, sai khiến, mệnh lệnh.
Đệ quy giới hạn độ sâu 5 (食べられなかった cần lùi 3 bước). Giữ danh sách đã thử để chống lặp vô hạn.
Lọc ứng viên: chỉ giữ cái thực sự có trong từ điển.
Xong khi: thử 20 dạng chia, đúng ít nhất 15. Ghi vào NOTES.md.

Tối 23/8 — Dọn dẹp + kiểm tra bộ nhớ

Không có unit test thì AddressSanitizer là lưới an toàn duy nhất.

```
target_compile_options(dict-core PRIVATE
  $<$<CONFIG:Debug>:-fsanitize=address -fsanitize=leak>)
target_link_options(dict-core PUBLIC
  $<$<CONFIG:Debug>:-fsanitize=address -fsanitize=leak>)
```

Chạy importer đủ 14.657 dòng, rồi chạy app tra vài chục từ, thêm/bỏ yêu thích.

- Mọi malloc có free chưa? Mọi nhánh lỗi có dọn trước khi return chưa?
- File .c nào còn #include <Q...>?
- Xoá core/entry.h, core/kana.*, core/dictionary.* nếu chưa xoá
- .gitignore: thư mục build, *.db, *.csv
- Clone về thư mục mới thử build

Tối 24/8 — Đóng gói + README

- Build Release, windeployqt, thử trên máy chưa cài Qt
- README: mô tả 1 dòng, ảnh chụp màn hình, **sơ đồ ranh giới C/C++**, hướng dẫn build, nguồn dữ liệu
- Nêu lý do để .db bên ngoài binary, và lý do tách user.db khỏi dictionary.db
- Nêu rõ ranh giới: lỗi runtime là C, công cụ build-time là C++

Tối 25, 26, 27/8 — Dự phòng (ba tối)

Chắc chắn có việc bị trễ. Ba tối này dành cho nó.

Nếu không nợ gì, theo thứ tự ưu tiên:

- dict_csv.c — chuyển nốt importer sang C, nếu thấy muốn toàn bộ là C
- Unit test cho dict_utf8 và dict_deinflect — C thuần với assert.h, không cần framework
- Flashcard ôn tập từ danh sách yêu thích
- Luồng nền QtConcurrent cho truy vấn

7. Thứ tự cắt nếu bị chậm

Ưu tiên	Hạng mục
1 (không cắt)	Chuyển lỗi sang C, app chạy được
2	Tìm kiếm tức thời + thông minh
3	Màn hình chi tiết + sao chép
4	Đóng gói + README
5	Dọn dẹp + kiểm tra bộ nhớ
6	Lịch sử + yêu thích
7	Deinflection phần 1
8	Deinflection phần 2
9	Bộ lọc + duyệt theo cấp

Với ba tối dự phòng, khả năng phải cắt là thấp.

8. Nghiệm thu cuối

- [] Không file `.c` nào include header Qt
- [] `find_package` không còn `Sql` — toàn dự án không dùng Qt SQL
- [] `COUNT(*)` = 14.657, không dòng nào `reading` rỗng
- [] `EXPLAIN QUERY PLAN` không câu nào hiện `SCAN words`
- [] `searchByMeaning("yêu")` ra kết quả — FTS5 đã được nạp
- [] Tắt app mở lại, lịch sử và yêu thích còn nguyên
- [] `dictionary.db` không bị ghi vào khi dùng app
- [] `AddressSanitizer` sạch khi chạy importer đủ 14.657 dòng
- [] Clone về thư mục mới build được
- [] Người khác giải nén file release là chạy được

9. Câu trả lời phỏng vấn

"Vì sao tách lỗi C khỏi tầng Qt?"

Lỗi không phụ thuộc framework nên đem sang nền tảng khác được. Tầng C++ chỉ làm một việc: chuyển `DictEntry` sang `QVector<Entry>` cho model QML. Em kiểm tra ranh giới bằng một quy tắc: không file `.c` nào được include header Qt, và `dict-core` không link Qt gì cả.

"Vì sao importer vẫn là C++?"

Nó là công cụ build-time, chạy một lần để sinh file `.db`, không đóng gói kèm sản phẩm. Phần đọc CSV dùng Qt cho nhanh, nhưng nó vẫn ghi database qua đúng API C mà app dùng — nên API đó được kiểm chứng bởi cả hai đường.

"Xử lý Unicode trong C thế nào?"

`QString` lưu UTF-16 nên mỗi kana là một `QChar`, trừ `0x60` là xong. Sang C thì chuỗi là mảng byte UTF-8, kana chiếm 3 byte, nên em viết `utf8_decode/utf8_encode` — đọc byte đầu để biết độ dài, ghép bit thành codepoint, tính toán, mã hoá ngược.

"Vì sao struct dùng mảng cố định mà không phải `char*`?"

Em đo độ dài thực tế trên dữ liệu trước: `word` dài nhất 42 byte, `english` 145 byte. Nếu mỗi trường là `char*` thì 100 kết quả cần 800 lần `malloc/free`, mỗi lần là một cơ hội rò rỉ. Mảng cố định thì cả danh sách là một `malloc`, một `free`. Cái giá là ~600 byte mỗi entry, với 100 kết quả là 60KB.

"Quản lý bộ nhớ ra sao khi không có RAI?"

Quy ước sở hữu ghi rõ trong header: hàm nào cấp phát thì có hàm `_free` đi kèm, kể cả khi hàm trả về lỗi. Build Debug bật `AddressSanitizer`, chạy importer 14.657 dòng để bắt leak.

"Chỗ nào biết là chưa tốt?"

Ba chỗ: `DictEntryList` chưa cấm copy nên gán qua lại sẽ double-free; struct dùng mảng cố định nên `meaning` dài quá 128 byte bị cắt — chưa gặp trong dữ liệu hiện tại nhưng em đã thêm cảnh báo lúc import; và importer chưa chuyển sang C.